



دليل تقني وتجربة مستخدم

واجهات وتقنيات تطبيق Work Key Mobile

شرح عملي لكل واجهة على حدة: الهدف، رحلة المستخدم، إدارة الحالة، اتصال الباك، المكتبات المستخدمة، وأهم القرارات التصميمية والأمنية.

تاريخ التوثيق

22-08-2026

إصدار التطبيق

1.0.0+1

إطار العمل

Flutter 3.47

المحتويات

01	فكرة التطبيق والبنية العامة	17	التقديم على وظيفة
02	التقنيات والمكتبات	18	طلباتي
03	الأنظمة المشتركة	19	تفاصيل الطلب
04	واجهة البداية Splash	20	طلب معلومات إضافية
05	التعريف بالتطبيق	21	النشاط
06	تسجيل الدخول	22	الإشعارات
07	إنشاء حساب	23	الملف الشخصي
08	نسيان كلمة المرور	24	تحرير بيانات الملف
09	التحقق من الرمز	25	ملفات CV
10	تغيير كلمة المرور	26	مراجعة CV وتعديله
11	الغلاف والتنقل السفلي	27	الامتحانات
12	الرئيسية للزائر	28	جلسة الامتحان المحمية
13	الرئيسية للمستخدم	29	المقابلات
14	استكشاف الوظائف والفلاتر	30	مكالمة الفيديو داخل التطبيق
15	فرص الشركة	31	خريطة أهم مسارات API
16	تفاصيل الوظيفة	32	الجودة والاختبارات

نطاق الدليل: هذا المستند مبني على كود تطبيق الهاتف الحالي، ويشرح تجربة الباحث عن عمل. بعض مسارات صاحب العمل موجودة في طبقة API لكنها ليست محور الواجهات الحالية.

فكرة التطبيق والبنية العامة

Work Key منصة توظيف للباحث عن عمل. تجمع اكتشاف الفرص، التقديم، إدارة الطلبات، تحليل السيرة الذاتية، الامتحانات والمقابلات داخل تطبيق واحد ثنائي اللغة.



إدارة الحالة

لكل مجال Cubit مستقل: المصادقة، الوظائف، الملف، CV، الطلبات، الامتحانات، المقابلات والإشعارات.

طبقة العرض

كل شاشة مسؤولة عن العرض والتفاعل فقط، وتستخدم ألوان ThemeData ونصوص context.tr لتتأقلم مع الوضع واللغة.

طبقة الشبكة

RemoteApi يضيف التوكن واللغة، يضبط المهلة، يعالج multipart، ويوحد تسجيل الأخطاء.

طبقة المستودع

تحوّل ردود الباك إلى Models مفهومة، وتوحد اختلافات شكل البيانات، وتمنع الواجهة من الاعتماد على JSON خام.

مبادئ التصميم الرئيسية

وصول آمن

التوكن، التحقق من البريد، منع اللقطات، ورصد مغادرة الجلسات الحساسة.

مصدر حقيقة واحد

الباك هو المرجع النهائي للحالة، والواجهة تحدّث نفسها بعد كل إجراء.

حالة واضحة

Loading, Empty, Error, Success مع Retry موحد.

أهم فكرة: التطبيق ليس مجموعة صفحات منفصلة؛ إنه سلسلة حالات مترابطة. نجاح تحديث CV مثلاً يجب أن ينعكس على الملف الشخصي ثم على أهلية التقديم، لذلك تم فصل المسؤوليات مع إبقاء تحديث الحالة بين الوحدات واضحاً.

التقنيات والمكتبات المستخدمة

المكتبة	الإصدار	الاستخدام الفعلي
Flutter / Dart	3.13 / 3.47	بناء الواجهات، Material 3، الأنيميشن، إدارة دورة حياة التطبيق.
flutter_bloc	9.1.1	Cubit و BlocBuilder و BlocConsumer لكل حالات التطبيق.
dio	5.11.0	طلبات multipart، REST، تنزيل bytes، المهلات ومعالجة الأخطاء.
shared_preferences	2.5.5	حفظ التوكن، اللغة، الثيم وحالة onboarding.
firebase_core / messaging	16.5.0 / 4.13.0	تهيئة Firebase واستقبال FCM داخل التطبيق وخارجه.
flutter_local_notifications	22.3.0	عرض إشعار محلي عندما تكون الرسالة واردة والتطبيق مفتوح.
livekit_client	2.11.0	غرفة مكالمة الفيديو، الصوت والكاميرا داخل التطبيق.
file_picker	12.0.0	رفع CV، مرفقات طلب المعلومات، إجابات الملفات، وحفظ CV على الجهاز.
image_picker	1.2.3	اختيار أو تصوير صورة الملف الشخصي.
pin_code_fields	9.4.0	حقل OTP مؤلف من ست خانات مع حركة إدخال.
intl	0.20.3	تنسيق مواعيد الوظائف والطلبات والإشعارات والمقابلات.
smooth_page_indicator	2.0.1	مؤشر صفحات onboarding.
flutter_holo_date_picker	2.0.0	منتقي تاريخ مشترك في مكونات الإدخال.
flutter_radio_group	2.1.0	مجموعات خيارات ضمن المكونات المشتركة.
hexcolor	3.1.3	قراءة بعض قيم الألوان القديمة ضمن constants.

تبعيات موجودة وليست مستخدمة مباشرة في واجهات **lib** الحالية: `equatable` و `google_nav_bar` و `dropdown_button2` و `font_awesome_flutter` و `url_launcher`. وجودها لا يعني أنها جزء من التدفق الحالي، ويمكن تقييم حذفها لاحقًا لتقليل الصيانة.

التقنيات الأصلية من Flutter

MethodChannel

PopScope

AnimatedSwitcher

TweenAnimationBuilder

AnimationController

Navigator

Material 3

WidgetsBindingObserver

الثيم، اللغة، الشبكة والإشعارات

العربية والإنكليزية

ملفا `en.json` و `assets/translations/ar.json` يحملان المفاتيح نفسها. `LocaleCubit` يحفظ الاختيار، ويُرسل `Accept-Language` و `Content-Language` مع كل طلب.

Light / Dark Theme

`AppThemeData` ييني `ColorScheme` منفصلاً للوضعين، ويغطي `Scaffold`، `Card`، `BottomSheet`، `Dialog`، `Chip`، `SnackBar` الوضع الافتراضي `Light`.

الإشعارات

`FCM token` يُرسل إلى `/device-tokens` مع المنصة واللغة، ويُعاد تسجيله عند `refresh` أو `resume`. الإشعار داخل التطبيق يتحول إلى `local notification`.

الوسائط

`media_url.dart` يحوّل المسارات التي يعيدها الباك إلى روابط قابلة للعرض، وتستخدم `CompanyLogo` و `Image.network` حالات `fallback` عند فشل الصورة.

التحميل والحالات الفارغة

`Skeletons` و `CircularProgressIndicator` للتحميل، رسائل `Empty` عند غياب البيانات، وزر `ModernRetryButton` عند فشل قابل لإعادة المحاولة.

الأخطاء

`DioErrorHandler` يفرّق بين `timeout`، انقطاع الشبكة، خطأ الباك والجلسة. `AppSnackBar` يعرض رسالة عصرية متوافقة مع الثيم.

الفكرة المهمة: لا تُترجم الأزرار فقط؛ اللغة المختارة تصل إلى الباك أيضًا، لذلك يمكن للخادم إعادة `labels` ورسائل وحالات مترجمة بنفس لغة الهاتف.

تعرض الهوية البصرية بحركة قصيرة، ثم تقرر هل يذهب المستخدم إلى onboarding أو تسجيل الدخول أو التطبيق الرئيسي.

التقنيات

AnimationController, Fade/Scale, SharedPreferences, Navigator.pushReplacement

الملف

lib/screens/auth/SplashScreen.dart

المسؤولية

قراءة القيم المحفوظة والتأكد من وجود جلسة.

الحالة

لا تحتاج Cubit مستقلاً؛ القرار محلي وسريع.

التدفق



أهم فكرة: استخدام pushReplacement يمنع الرجوع بزر Back إلى شاشة البداية بعد دخول التطبيق.

صفحات أفقية مختصرة تقدم الوظائف والملف المهني ومسار التوظيف، مع مؤشر حركة وأزرار تخطي/متابعة.

الحفظ والتنقل

عند الإنهاء تُحفظ علامة onboarding في SharedPreferences ثم تُفتح شاشة تسجيل الدخول؛ وبذلك لا تظهر الجولة في كل تشغيل.

العناصر

- PageView للمحتوى.
- صور من assets.
- SmoothPageIndicator.
- زر متابعة يتغير في الصفحة الأخيرة.

lib/screens/auth/onboardind_screen/onboarding_screen.dart • onboarding_page.dart

أهم فكرة UX: المحتوى قصير ومرئي، والتقدم واضح؛ الهدف ليس شرح كل ميزة بل إعطاء سبب مقنع للمتابعة.

حقل البريد وكلمة المرور، استعادة كلمة المرور، دخول كزائر، وإنشاء حساب. جميع العناوين والرسائل مترجمة وتستجيب للوضع الداكن.

استجابة الباك

عند النجاح يُحفظ token. عند POST auth/login .
EMAIL_NOT_VERIFIED ينتقل المستخدم تلقائيًا إلى شاشة OTP ويُرسل رمز جديد.

التحقق المحلي

يُمنع الإرسال إذا كان أحد الحقليين فارغًا، وتظهر AppSnackBar مترجمة بدل خطأ خام.

المكتبات

flutter_bloc, shared_preferences, Dio, AppLocalizations, مكونات الحقول المشتركة.

إدارة الحالة

AuthCubit: Loading يعطل الزر، Success ينقل إلى Layout, Error يعرض الرسالة أو يغيّر المسار للتحقق.

lib/screens/auth/login/login_screen.dart • data/repo/auth_repo.dart

أهم فكرة: عدم معاملة الحساب غير المؤتق كخطأ نهائي؛ يتم تحويله إلى مسار قابل للإكمال مع الاحتفاظ بالبريد وكلمة المرور اللازمة لتسجيل الدخول بعد التحقق.

تجمع الاسم والبريد والهاتف والمدينة وتفاصيل الموقع وكلمة المرور وتأكيدها، ثم تنتقل المستخدم إلى OTP.

التحقق

الحقول المطلوبة وتطابق كلمتي المرور تُفحص محليًا قبل
POST auth/register/job-seeker .

الحقول

المدينة تأتي من ReferenceRepo، مع loader داخل
Dropdown. يتم إرسال terms_accepted: true وفق عقد التسجيل.

الترجمة

العنوان، الحقول، رسائل النقص والتطابق، زر الزائر وروابط
الدخول كلها تعتمد مفاتيح auth في ملفي الترجمة.

الانتقال

عند تُفتح VerificationCodeScreen بنفس البريد وكلمة
المرور. التسجيل نفسه يرسل الرمز الأول، لذلك لا يُكرر الطلب
عند فتح الصفحة.

lib/screens/auth/register/register_screen.dart

أهم فكرة: فصل إنشاء الحساب عن إنشاء الجلسة؛ لا يُحفظ token إلا بعد أن يصبح البريد متحققًا.

واجهة بسيطة تطلب البريد فقط، ترسل رمز الاستعادة، ثم تنتقل إلى شاشة التحقق العامة بوضع `resetPassword`.

الحالة

.AuthLoading, ForgotPasswordSuccess, AuthError

API

POST auth/forgot-password

الملف

lib/screens/auth/forgot_password/forgot_password_screen.dart

التصميم

رسم توضيحي، حقل واحد، CTA واضح، رسائل مترجمة.

أهم فكرة: إبقاء استعادة كلمة المرور في واجهتين لاحقتين واضحتين: التحقق أولاً، ثم اختيار كلمة جديدة، بدل مزج الخطوات في نموذج طويل.

تُستخدم بعد التسجيل، عند محاولة دخول حساب غير متحقق، وبعد نسيان كلمة المرور. تعرض بريدًا مخفيًا جزئيًا وعدّاد إعادة الإرسال.

إعادة الإرسال

يحتاج البريد فقط. أزيل شرط `POST auth/email/resend-otp` كلمة المرور الذي كان يُظهر رسالة «أعد تسجيل الدخول» بصورة خاطئة.

إدخال الرمز

MaterialPinField من `pin_code_fields` ست خانات LTR، حركة `scale`، وإرسال تلقائي عند اكتمال حساب التفعيل.

استعادة كلمة المرور

بعد إدخال الرمز ينتقل إلى `ResetPasswordScreen` ويُمرّر البريد والرمز دون تسجيل دخول.

تحقق الحساب

`POST auth/email/verify-otp` ثم `login` بنفس البيانات للحصول على `token`، ولا يمكن الرجوع من مسار تفعيل الحساب قبل اكتماله.

lib/screens/auth/verification/verification_screen.dart

أهم فكرة: إعادة استخدام واجهة واحدة مع `enum` واضح للتدفق يمنع اختلاف السلوك والتصميم بين التسجيل والدخول والاستعادة.

واجهة تعيين كلمة مرور جديدة

10

المرحلة النهائية من الاستعادة.

حقل كلمة المرور والتأكيد ضمن بطاقة واضحة، مع شرط ثمانية أحرف ورسائل تطابق وحالة تحميل.

Payload

.email, otp, password, password_confirmation

API

POST auth/reset-password

التييم

ألوان النص والحقل والرمز من ColorScheme في الوضعين.

النجاح

رسالة نجاح ثم إزالة المسار والعودة إلى Login.

أهم فكرة: كلمة المرور لا تُغيّر عند إدخال OTP فقط؛ يبقى التحقق منفصلاً عن الحفظ حتى تكون رحلة المستخدم مفهومة وقابلة للتصحيح.

Header متحرك دون AppBar تقليدي، و Bottom Navigation مخصص بخمس جهات؛ زر الرئيسية هو شعار Work Key في المنتصف بحركة خاصة.

التحميل الكسول

لا تُبنى كل الصفحات الثقيلة فورًا؛ تُسجل التبويبات التي زارها المستخدم لتقليل العمل الأولي.

التبويبات

استكشاف، طلباتي، الرئيسية، النشاط، الملف الشخصي.
تستخدم IndexedStack للحفاظ على حالة الشاشة عند الانتقال.

التوافق

SafeArea، قياسات مرنة، labels مترجمة، padding خاص لتبويب الملف، وألوان مستقلة للدارك واللايت.

الأنيميشن

AnimatedContainer و AnimationController و AnimatedOpacity و Transform لتحريك الشعار المركزي وحالة الاختيار والهيذر.

lib/layout/layout.dart • shared/components/animated_app_logo.dart

أهم فكرة: الرئيسية ليست مجرد icon إضافي؛ الشعار نفسه أصبح نقطة الارتكاز البصرية والوظيفية في منتصف التنقل.

تعرض Hero، وظائف وشركات ومزايا المنصة. يمكن للزائر فتح تفاصيل وظيفة، لكن الضغط على التقديم ينقله إلى تسجيل الدخول بدل إظهار خطأ API.

المسارات

التسجيل، الدخول، تفاصيل الوظيفة، فرص الشركة، واستكشاف المزيد.

مصدر البيانات

GET home مع auth اختياري؛ الباك يحدد viewer والكتل المناسبة.

الصور

عرض cover/logo للشركة من الرابط الذي يعيده الباك مع fallback يحمل الحروف الأولى.

الحالات

Skeleton داكن/فاتح، Empty مفيد دون زر Retry عند عدم وجود بيانات، و Retry فقط عند فشل الشبكة.

أهم فكرة UX: السماح بالمشاهدة قبل التسجيل يقلل الاحتكاك، بينما تحويل Apply إلى Login يحافظ على وضوح السبب والإجراء التالي.

ترحيب بالاسم، جرس إشعارات مع العدد غير المقروء، اكتمال الملف، إجراءات مطلوبة، وظائف موصى بها وشركات.

الإجراءات الذكية

HomeNavigation يقرأ نوع الهدف ومعرفه ويوجه إلى وظيفة أو طلب أو اختبار أو مقابلة أو طلب معلومات.

HomeCubit

يجلب HomeResponseModel ويحوّل كتل الباك إلى أقسام قابلة للعرض.

التحديث

RefreshIndicator يعيد تحميل البيانات، والإشعارات تُحدّث العدد في الهيدر.

الملف المهني

بطاقة نسبة الاكتمال تعرض العنصر التالي الناقص بدل نسبة بلا معنى.

lib/screens/home_screen/home_screen.dart • home_navigation.dart

أهم فكرة: الرئيسية تعمل كلوحة قرار، لا كقائمة محتوى؛ كل بطاقة يجب أن تقود إلى إجراء فعلي لا إلى صفحة فارغة.

تبويبات «لك، الأحدث، الكل»، بحث مع pagination، debounce، وفلاتر ديناميكية تأتي من schema الباك.

الترشيح

يعيد نسبة التطابق، وتظهر على `GET jobs/recommended` كارد الوظيفة المقترحة.

الفلتر

يحدد الحقول والأنواع `GET reference/job-filters` والخيارات؛ المصادر البعيدة تُحمّل عند الحاجة وباللغة المختارة.

الأداء

تأخير البحث يمنع طلبًا مع كل حرف، والصفحات تُدمج بدل استبدال القائمة.

حالة الوظيفة

Badges واضحة: جديدة، انتهى التقديم، تم التقديم. AppliedJobsStore يعزز الحالة من طلبات المستخدم.

lib/screens/explore_jobs • logic/explore_jobs_cubit

أهم فكرة: الفلاتر لا تُثبت داخل التطبيق؛ schema الباك يسمح بإضافة خيار أو تغيير endpoint دون إعادة بناء الواجهة كلها.

عند الضغط على كارد شركة يُفتح ExploreJobsScreen في وضع company_id، مع هوية الشركة وقائمة فرصها فقط.

الوسائط

Cover و Logo مستقلان؛ لا يُستخدم أحدهما بدل الآخر إلا كـ fallback.

إعادة الاستخدام

نفس كارد الوظيفة والتصفح مع query مختلف.

API

GET companies/{companyId}/jobs

الحالات

Loading، Empty، Error + Retry، خاص بالشركة.

أهم فكرة: إعادة استخدام شاشة الاستكشاف في وضع مقيد تقلل تكرار الكود وتحافظ على نفس تجربة البطاقات والفلاتر.

Hero ملون يحمل الشركة والمسمى والموقع والنمط والدوام، ثم الوصف والمهارات والمسؤوليات والمتطلبات والمزايا والموعد النهائي.

الشركة

CompanyLogo يعرض رابط logo، وصورة الغلاف تستخدم cover_image_url مع fallback بصري.

API

مع GET jobs/{id} مع preserveDioError لتمييز وظيفة محذوفة عن انقطاع شبكة.

الحالة المحلية

AppliedJobsStore يمنع إعادة التقديم حتى لو لم يتضمن رد تفاصيل الوظيفة viewer_application.

حالة الزر

Applied يعطل التقديم، Closed/Expired يعرض انتهاء التقديم، والزائر يُوجّه إلى Login.

lib/screens/jobs/job_details_screen.dart

أهم فكرة: نص زر التقديم نفسه يشرح الحالة الحالية، فلا يحتاج المستخدم لاكتشاف المنع بعد إرسال طلب فاشل.

Bottom Sheet حديث يوضح أن الملف الشخصي الموثق وآخر CV فعال سيشارك، ويطلب Cover Letter وأسئلة الفرز عند وجودها.

لا يوجد اختيار CV

الباك يعتمد معلومات البروفايل وCV الفعال، لذلك أزيل الاختيار اليدوي.

API

POST jobs/{id}/applications

الأساسي Payload

consent_to_share_profile: true و cover_letter

وإجابات الأسئلة المطلوبة.

منع التكرار

استجابة already applied تتحول إلى حالة واضحة وتُحفظ محليًا.

lib/screens/jobs/apply_job_sheet.dart

أهم فكرة: النموذج يرسل فقط ما يطلبه عقد الباك. كل حقل إضافي غير مدعوم يزيد فرص 422 دون قيمة للمستخدم.

بحث، فلاتر للحالات، عداد للطلبات التي تحتاج إجراء، وبطاقات تعرض الوظيفة والشركة والحالة والإجراء التالي.

التقسيم

الكل، نشطة، تحتاج إجراء، مكتملة. Empty message مستقل لكل مجموعة.

API

GET applications/my مع page وfilters.

التنقل

الكارد يفتح التفاصيل، والإجراء السريع قد يقود إلى اختبار أو مقابلة أو طلب معلومات.

البيانات

ApplicationsResponseModel يعالج pagination والعدادات وallowed actions.

أهم فكرة: ترتيب الطلبات حسب حاجتها لتصرف المستخدم أهم من ترتيبها الزمني فقط.

تعرض الوظيفة والشركة والحالة الحالية و Cover Letter وإجابات الفرز والتاريخ المتسلسل والتصرف التالي المتاح.

السحب

مع سبب اختياري وبعد
POST applications/{id}/withdraw
تأكيد Dialog.

الهوية

ApplicationStatusTheme يوحد لون وأيقونة كل حالة.

API

GET applications/{id}

الإجراءات

فتح طلب المعلومات، الاختبار، أو المقابلة بحسب
next_action.

أهم فكرة: الخط الزمني يحول الحالة التقنية إلى قصة مفهومة: ماذا حدث، متى، وما المطلوب الآن.

تعرض سؤال الشركة والموعد والحالة، وتسمح بكتابة الرد وإرفاق عدة ملفات ثم إرسال multipart.

الإرسال

عبر `POST information-requests/{id}/respond` مع `FormData`، مع `text` ومرفقات.

ال جلب

لعرض التفاصيل الحالية. `GET information-requests/{id}`

UX

تظهر أسماء الملفات ويمكن حذف كل مرفق قبل الإرسال، مع تعطيل الزر أثناء الرفع.

المكتبة

`file_picker` لاختيار الملفات و `Dio MultipartFile` لإرسالها.

أهم فكرة: نقل الإجراء كاملاً إلى الموبايل؛ الإشعار لا يكتفي بإخبار المستخدم بل يقوده إلى شاشة يستطيع فيها إنهاء المطلوب.

تعرض أحداث الطلبات والاختبارات والمقابلات والإشعارات بترتيب زمني، مع فلاتر وحالة غير مقروء.

التصفية

نوع الحدث، القراءة، الفرز والاتجاه.

التنسيق

intl للتاريخ وActivityStrings للنصوص.

API

مع query ديناميكي. GET activity

التفاعل

فتح الهدف وتحديد الإشعار كمقروء.

أهم فكرة: النشاط يجمع مصادر متعددة في نموذج بصري واحد، بينما تبقى وجهة كل حدث محددة ببيانات action من الباك.

AppBar مرّن يحمل أيقونة وعنوانًا لا يفيض في العربية، وزر «تحديد الكل كمقروء» أصبح أيقونة ذات Tooltip لتجنب خربطة المساحة العلوية.

التفاعل

عند الضغط يُحدد الإشعار كمقروء ثم يحلّل `action.type` و `target.id` و `route` لفتح المكان المقصود.

الحالات

المقروء وغير المقروء يختلفان في الخلفية والوزن والنقطة، مع `Empty` و `Error` و `Pull to refresh`.

التقنيات

`NotificationsCubit`, `intl`, `HomeNavigation`, `FirebaseMessaging` للإشعارات الخارجية.

API

`.list`, `unread-count`, `read`, `read-all`, `delete`

`lib/screens/notifications/notifications_screen.dart` • `services/push_notification_service.dart`

أهم فكرة: الإشعار الناجح ليس رسالة فقط؛ يجب أن يكون رابطًا موثوقًا إلى العنصر الذي تسبب به.

أصبح قسم CV أول عنصر بعد عنوان الصفحة، ثم صورة وبيانات المستخدم، الملخص، التواصل، التوفر، المهارات، الخبرات، التعليم والمقابلات.

الصورة

image_picker من المعرض أو الكاميرا، ثم multipart بالحقل image ، مع حذف مستقل.

API

GET profile ، ويحوّل ProfileRepo الأشكال المتداخلة إلى ProfileModel موحد.

الإعدادات

اللغة، الثيم، تسجيل الخروج من الجهاز أو كل الأجهزة.

تحديث CV

بعد التأكيد النهائي يطبق ProfileCubit البيانات المرجعة فورًا ثم يعيد GET profile لضمان التطابق.

أهم فكرة: ترتيب CV أولاً يضع الإجراء الأعلى قيمة في بداية الصفحة، بينما تبقى باقي البيانات قابلة للتعديل اليدوي.

Bottom Sheets متوافقة مع لوحة المفاتيح والثيم لتحرير الاسم والعنوان والملخص والموقع والتوفر والروابط، وإدارة الخبرة والتعليم والمهارات.

الخبرات

CRUD عبر `profile/experiences` مع إلزام المسمى واسم الشركة.

الملف الأساسي

ويعتمد التطبيق الرد الكامل مباشرة لتجنب `PUT profile` GET ثاني قد يفشل بعد نجاح الحفظ.

المهارات

تحميل `catalog` ثم `attach/detach` كل مهارة بمعرفها.

التعليم

CRUD عبر `profile/education` مع إلزام المؤسسة.

أهم فكرة: النماذج ترسل `payload` مطابقًا لعقد الباك بدل إرسال `Model` كامل قد يحتوي حقول قراءة فقط.

رفع PDF/DOC/DOCX، متابعة المعالجة، فتح المراجعة، تنزيل الملف إلى الجهاز، وحذف كل CV قابل للإلغاء على حدة.

التنزيل

ثم `GET cv/{id}/download` بصيغة bytes، ثم `FilePicker.saveFile` يفتح حفظ النظام ويكتب البيانات في الوجهة التي يختارها المستخدم.

الرفع

ثم `file_picker` ثم `Dio FormData` إلى `POST cv/upload` مع `.version_label`

دورة الحياة

`uploaded`، `processing`، `review`، `confirm`، `completed`؛
وتُخفى سجلات `audit` الملغاة أو المنتهية أو غير المتاحة.

الحذف

بعد `Dialog`؛ كل ملف يملك `busy` `POST cv/{id}/cancel` state خاصًا به.

lib/screens/profile/widgets/profile_cv_section.dart • data/api/cv_api.dart

أهم فكرة: التنزيل يتم بطلب مصادق عليه عبر Dio، لا بفتح رابط عام؛ بذلك يبقى ملف CV الخاص محميًا بالتوكن.

خمس خطوات: شخصية، خبرة، تعليم، مهارات، مراجعة نهائية. تعرض كل بطاقة القيمة الحالية والقيمة المستخرجة بتنسيق بصري، وتسمح بالإبقاء أو الاستخدام أو التعديل اليدوي.

التعديل اليدوي

Bottom Sheet ديناميكي: الهاتف/الملخص/الموقع، حقول الخبرة المطلوبة، التعليم، أو اسم المهارة. يتحقق من الحقول الإلزامية قبل الإرسال.

القرارات الفردية

Accept أو Reject لكل suggestion. عند التعديل تُرسل خريطة `edited_value` المطابقة لنوع الاقتراح.

التأكيد

لا يتفعل إلا عندما يقول الباك `can_confirm` ويصبح `POST cv/{id}/confirm`، ثم `unresolved = 0`

Bulk

استخدام كل اقتراحات القسم أو إبقاء الحالي دفعة واحدة عبر `cv/{id}/suggestions/decisions`

لماذا لا نعدل draft كاملاً مباشرة؟

عقد الباك الحالي يصرح بـ `can_edit_draft: false` ويمنع `updateReviewDraft`، لكنه يتيح `can_edit` لكل اقتراح وقواعد دقيقة لـ `edited_value`؛ لذلك استُخدم المسار المدعوم فعلياً.

lib/screens/profile/cv_review_screen.dart • data/api/cv_api.dart

أهم فكرة: لا يُحدّث الملف عند كل اختيار صغير؛ تُحفظ القرارات أولاً ثم يُطبّق كل شيء في Transaction واحدة عند التأكيد النهائي، ما يمنع ملفاً نصف محدّث.

قائمة الامتحانات المسندة، الحالة والموعد والمحاولات، ثم شاشة تفاصيل توضح التعليمات والمدة وتطلب تأكيدًا قبل بدء الجلسة.

Models

TestAssignmentModel يتعامل مع test assignment والمحاولة والنتيجة والحالات المحلية.

APIs

GET tests/{id} ، GET my/tests
POST tests/{assignmentId}/start

التنقل

لا تُفتح شاشة المحاولة حتى يعيد الباك attempt صالحًا وأسئلة.

الحالات

لا يظهر Retry عند غياب التكليفات؛ يظهر فقط عند فشل الشبكة. تواريخ العرض عبر intl.

أهم فكرة: فصل «قراءة التعليمات» عن «بدء العداد» يمنع بدء الامتحان عرضيًا.

تعرض الأسئلة والخيارات والنصوص والملفات، تحفظ الإجابات، وتراقب مغادرة التطبيق. الخروج يؤدي إلى forfeit وفق قواعد المنصة.

منع اللقطات

SecureSessionService يستدعي MethodChannel، MainActivity يفَعّل FLAG_SECURE.

حفظ الإجابات

POST test-attempts/{id}/answers/bulk ، ورفع الملف
بمسار السؤال، ثم submit بتأكيد نهائي.

منع الرجوع

PopScope يمنع إغلاق الجلسة بالطريقة العادية دون تطبيق منطق الإنهاء.

رصد الخروج

WidgetsBindingObserver يراقب
inactive/paused/detached ويرسل
POST tests/{assignment}/forfeit بسبب
.app_backgrounded

lib/screens/tests/test_attempt_screen.dart • services/secure_session_service.dart • MainActivity.kt

أهم فكرة أمنية: الحماية ليست واجهة تحذير فقط؛ هناك علم نظام يمنع الالتقاط وحدث lifecycle يُبلغ الباك عند مغادرة الجلسة.

قائمة مقابلات المستخدم مع المواعيد والحالات، ثم تفاصيل نوع المقابلة ومدتها وموقعها وحضورها والوظيفة والشركة.

التوقيت

تحويل وقت الباك إلى local time وعرض التاريخ والساعة والمدة.

APIs

GET interviews/{id} ، GET my/interviews
POST interviews/{id}/confirm

الفيديو

للمقابلة الرقمية يُطلب video-session ثم تُفتح واجهة LiveKit داخل التطبيق.

الحضور

زر التأكيد يظهر فقط عندما تسمح حالة المقابلة، ويتغير أثناء الطلب.

أهم فكرة: Eligibility الدخول يحدده الباك وفق الحالة والوقت، وليس مجرد وجود زر ثابت في الواجهة.

تتصل بالغرفة باستخدام URL و token من الباك، تعرض المشاركون المحلي والبعيد، وتوفر الكاميرا والميكروفون وإنهاء المكالمة دون فتح تطبيق خارجي.

المكتبة

livekit_client لإدارة Room و Participant و Tracks الصوتية والمرئية.

إنشاء الجلسة

يعيد بيانات POST interviews/{id}/video-session الاتصال الآمنة.

الخروج

Lifecycle observer يفصل الغرفة عند مغادرة التطبيق، PopScope يدير الإنهاء المتعمد.

الأمان

FLAG_SECURE يمنع screenshot/recording على Android أثناء الجلسة.

lib/screens/interviews/interview_video_screen.dart

أهم فكرة: استضافة المقابلة داخل التطبيق توّجّد تجربة المستخدم وتسمح بتطبيق سياسات الأمان ودورة الحياة التي لا يمكن ضمانها عند فتح رابط خارجي.

خريطة أهم مسارات API المستخدمة

المجال	المسارات الأساسية	الواجهة المستهلكة
المصادقة	auth/login, register/job-seeker, email/resend-otp, email/verify-otp, forgot-password, reset-password	Login / Register / OTP / Reset
الرئيسية	home	GuestHome / Home
الوظائف	jobs, jobs/{id}, jobs/recommended, companies/{id}/jobs, reference/job-filters	Explore / Company / Job Details
الطلبات	jobs/{id}/applications, applications/my, applications/{id}, withdraw	Apply / Applications / Details
معلومات إضافية	information-requests/{id}, respond	InformationRequestScreen
الملف	profile, profile/avatar, experiences, education, profile/skills	Profile / Edit Managers
CV	cv, upload, download, parsed, review, suggestions/decisions, suggestion accept/reject, confirm, cancel	CV Files / Review Wizard
الامتحان	my/tests, start, questions, answers/bulk, file, submit, forfeit	Tests / Test Attempt
المقابلة	my/interviews, interviews/{id}, confirm, video-session	Interviews / Video
الإشعارات	notifications, unread-count, read, read-all, delete, device-tokens	Header / Notifications / Push service
النشاط	activity	ActivityScreen

رؤوس الطلبات: Authorization Bearer token للطلبات الخاصة، Accept application/json، و Accept-Language/Content-Language حسب لغة التطبيق. تُزال Content-Type اليدوية عند FormData كي يولد Dio multipart boundary الصحيح.

الاختبارات والتحقق من النسخة

Flutter Test ✓

نجح 36 اختبارًا تشمل Models، عقود CV، تسلسل المراجعة، الثيم الداكن للمصادقة، الوظائف، المقابلات والرئيسية.

Flutter Analyze ✓

لم تظهر أخطاء أو تحذيرات مانعة للبناء. بقيت معلومات lint قديمة مثل print وواجهات deprecated في أجزاء أخرى من المشروع.

Translation JSON ✓

تم تحليل ملفي العربية والإنكليزية بنجاح، والتأكد من وجود المفاتيح الثابتة المستخدمة في auth وprofile وnotifications.

Android Debug Build ✓

اكتمل flutter build apk --debug ونتاج ملف APK صالح.

اختبارات مهمة موجودة

- عدم إرسال edited_value عند اختيار قيمة CV دون تعديل.
- إرسال edited_value كخريطة حقيقية عند التعديل اليدوي.
- عدم السماح بالتأكيد النهائي قبل إنهاء كل قرارات CV.
- قراءة viewer_application وحالة التقديم ونسب التطابق.
- عرض الشعار والنص والحقول بصورة صحيحة في الوضع الداكن.
- تحليل جلسة فيديو المقابلة والعقود المتداخلة بأمان.

ملاحظة مستقبلية: تحذير Built-in Kotlin يأتي من Flutter الحديث وبعض plugins مثل flutter_webrtc وlivekit_client. البناء الحالي ناجح، لكن يُفضل متابعة إصدارات المكتبات قبل ترقية Flutter التالية.

الخلاصة: أهم ما يميز التطبيق هو ربط رحلة التوظيف كاملة: ملف مهني قابل للبناء من CV ومراجعتة يدويًا، وظائف بحالات دقيقة، طلبات قابلة للمتابعة، ثم امتحان ومقابلة داخل بيئة محمية ومترجمة.